

JomCode Coding Bootcamp

Conditions and validation

Understand, predict, type, run, then practice.

What you will be able to do

- 01 Use comparisons, Boolean logic and if/elif/else.
- 02 Distinguish truthiness from explicit boundary validation.
- 03 Express capacity and role decisions with readable branches.

Plain meaning → technical view → use

PLAIN MEANING

A program chooses what to do when a condition changes.

TECHNICAL VIEW

Comparisons, Boolean operators and if/elif/else control execution paths.

WHEN YOU USE IT

Reject invalid capacity and decide whether a workshop has seats.

TIP

Try values exactly on each boundary, not just obvious valid values.

Worked example 1: Capacity boundaries



chapter_05_conditions_and_validation.py

```
1 capacity = 20
2 if 1 ≤ capacity ≤ 500:
3     print("Valid capacity")
4 else:
5     print("Capacity must be 1 to 500")
```

PREDICT Are 1 and 500 accepted?

Example 1: output and reason

TERMINAL	>_	EXPECTED OUTPUT
----------	----	-----------------

		Valid capacity
--	--	----------------

WHY THIS HAPPENS

Chained comparisons make both inclusive boundaries visible.

Exercise 1: Strict capacity

BUILD

Validate a value as an integer from 1 to 500, rejecting bool and numeric text.

CHECK

1/500 valid. 0/501/True/"5" invalid.

IF STUCK

Check the type before comparing values.

Worked example 2: Role and ownership



chapter_05_conditions_and_validation.py

```
1 role = "organizer"
2 owner_id = "u1"
3 current_user_id = "u2"
4 if role == "organizer" and owner_id == current_user_id:
5     print("Can edit")
6 else:
7     print("Forbidden")
```

PREDICT Is the role alone enough?

Example 2: output and reason

TERMINAL	>_	EXPECTED OUTPUT
		Forbidden

WHY THIS HAPPENS

Server-side authorization later checks both the actor role and the resource owner.

Exercise 2: Authorization decision

BUILD

A user may edit only when they are an organizer and own the workshop.

CHECK

organizer+owner allowed. participant+owner denied. organizer+other denied.

IF STUCK

Both requirements must hold.

Worked example 3: Explicit missing check



chapter_05_conditions_and_validation.py

```
1 value = 0
2 print(value is None)
3 print(not value)
4 optional = None
5 print(optional is None)
```

PREDICT Why can truthiness lose a meaningful zero?

Example 3: output and reason

TERMINAL	>_	EXPECTED OUTPUT
		False
		True
		True

WHY THIS HAPPENS

An absence check should use is None when zero is valid.

Exercise 3: Attendance message

BUILD

Choose Available, Full or Invalid for capacity 3 and count 4.

CHECK

3,2 => Available. 3,3 => Full. 3,4 => Invalid.

IF STUCK

Order mutually exclusive branches.

Checkpoint

PROVE IT

Capacity and authorization decisions include normal, boundary and invalid cases.